Distributed Real-Time Drawing Board with Mini-RAFT Consensus

Architecture Document Cloud Computing Project 2026

Name	Name
Deepthi M	PES1UG23AM092
Samrudhi Rao	PES1UG23AM915
Dhanush Shetty	PES1UG23AM097
Gaurav Kothari	PES1UG23AM106

---

1. System Overview

We built a distributed real-time collaborative drawing board where multiple users can draw on a shared canvas simultaneously. The backend is not a simple server — it is a cluster of three replica nodes that maintain a shared stroke log using a Mini-RAFT consensus protocol.

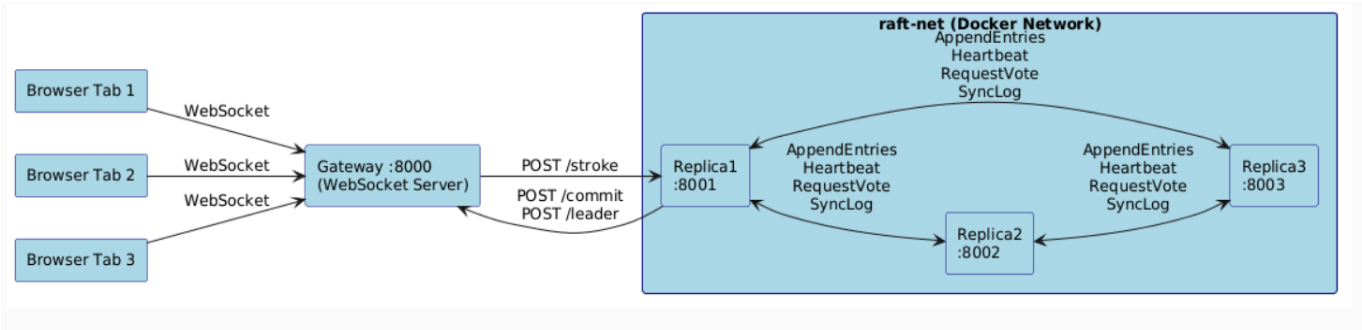
We used Python (FastAPI) for the backend, WebSockets for real-time communication, Docker and docker-compose for containerization, and a plain HTML/JavaScript canvas for the frontend.

The system guarantees that even if any replica crashes or restarts, the drawing board stays available and all users see the same canvas state.

---

2. Cluster Architecture

Our system has four Docker containers running on a shared Docker network called raft-net:



Gateway accepts all browser WebSocket connections, forwards strokes to the current leader, and broadcasts committed strokes back to all clients.

Replica1, Replica2, Replica3 are identical RAFT nodes. Each one can be a Follower, Candidate, or Leader depending on the election outcome. They communicate with each other using HTTP RPCs.

Each replica folder (replica1/, replica2/, replica3/) is bind-mounted into its container so that saving any file causes the container to auto-reload via uvicorn --reload.

---

3. Mini-RAFT Protocol Design

3.1 Node States

- **Follower** — Default state on startup. Waits for heartbeats from the leader. If no heartbeat arrives within the election timeout, it becomes a Candidate.
- **Candidate** — Increments the term, votes for itself, and sends RequestVote to both peers. Becomes Leader if it receives majority votes (2 out of 3).
- **Leader** — Handles all stroke replication. Sends AppendEntries to followers. Sends heartbeats every 150ms to prevent new elections.

3.2 Timing

Parameter	Value
Election timeout	Random 500–800 ms
Heartbeat interval	150 ms

Majority quorum	2 out of 3 nodes

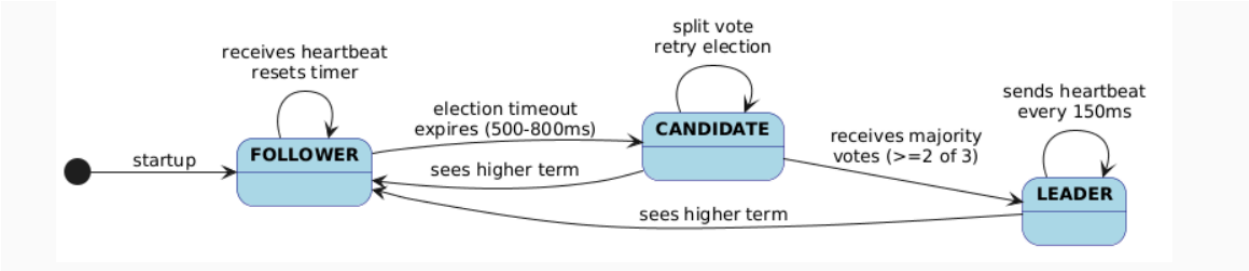
3.3 Log Replication Steps

When a user draws a stroke:

1. Browser sends stroke over WebSocket to Gateway
2. Gateway forwards stroke to current Leader via POST /stroke
3. Leader appends stroke to its local log with current term and index
4. Leader sends AppendEntries to both followers simultaneously
5. Followers append to their logs and reply with success
6. Leader commits the entry once 2 or more ACKs received
7. Leader calls POST /commit on Gateway with the committed entry
8. Gateway broadcasts stroke to all connected browser clients

---

4. State Transition Diagram



---

5. API Endpoints

Replica Endpoints

Endpoint	Method	What it does
/request-vote	POST	Candidate asks peer for a vote
/append-entries	POST	Leader replicates a log entry to followers

/heartbeat	POST	Leader sends periodic liveness signal
/sync-log	GET	Follower requests missing entries from leader
/stroke	POST	Gateway sends a new stroke to the leader
/status	GET	Returns role, term, log length, commit index

Gateway Endpoints

Endpoint	Method	What it does
/ws	WebSocket	Browser connects here to draw and receive strokes
/commit	POST	Leader pushes committed stroke to gateway
/leader	POST	Replica tells gateway who the new leader is
/status	GET	Returns connected clients, leader, stroke count

---

6. Failure Handling

Leader Failure

When the leader crashes, followers detect the missing heartbeat after 500–800ms. One follower starts an election, gets majority votes, becomes the new leader, and announces itself to the gateway. The gateway reroutes all traffic to the new leader. Total downtime is under 800ms.

Follower Failure

The leader keeps running because it still has majority (2 out of 3). When the crashed follower restarts it comes back with an empty log. On the first heartbeat it detects it is behind and calls GET /sync-log on the leader to fetch all missing committed entries. After syncing it participates normally.

Hot Reload

When we edit any file in replica1/, replica2/, or replica3/ and save it, uvicorn detects the change and gracefully restarts the container. If it was a follower, the leader keeps sending heartbeats and the follower rejoins after restart. If it was the leader, the other two nodes elect a new leader within 800ms and the restarted node rejoins as follower.

Split Vote

If two candidates start elections at the same time, neither may get majority. Both revert to follower and retry after a new random timeout. Because timeouts are randomised between 500–800ms they will not clash every time and a winner is elected quickly.

Safety Rules

- Committed entries are never overwritten — commit_index only moves forward
- Any node seeing a higher term immediately steps down
- Only one leader can exist per term because majority quorum is required
- Log consistency is enforced using prevLogIndex and prevLogTerm checks

---

7. Docker Setup

Container	Port	Role	Bind Mount
gateway	8000	WebSocket server	./gateway
replica1	8001	RAFT node	./replica1
replica2	8002	RAFT node	./replica2
replica3	8003	RAFT node	./replica3

All containers share a Docker bridge network called raft-net. Each replica gets its identity (REPLICA_ID) and peer addresses (PEER_REPLICA2 etc.) from environment variables in docker-compose.yml so the same Docker image runs all three replicas.

---

8. Real-World Relevance

Concept	What we built	Real world
Consensus	Mini-RAFT with term elections	etcd in Kubernetes, Consul
Leader failover	Auto re-election in under 800ms	Kubernetes control plane
Zero downtime deploy	Hot reload via bind mounts	Blue-green rolling updates
State replication	Append-only stroke log	CockroachDB, Kafka
Real-time collaboration	WebSocket fan-out	Figma, Miro, Google Docs
Service isolation	Docker + docker-compose	Kubernetes microservices